

IAM, Secrets, Configuration and Environment Boundaries

Who may act, what they may read, and where the walls are — for the service you already deployed

WHAT WE'LL COVER

Five chapters, three labs, two activities

01 Identity and the hierarchy

02 Identity for workloads

03 Secrets

04 Environment boundaries

05 Auditing access

01

Identity and the hierarchy

Who can act, what they may do, and where that permission comes from — principals, roles, inheritance and the guardrails above IAM.

01 Identity and the hierarchy

02 Identity for workloads

03 Secrets

04 Environment boundaries

05 Auditing access

Four kinds of principal — and only one of them is a person

Principal	Looks like	Is for	Authenticates with
User	user:priya@example.com	a person	Google sign-in, MFA
Group	group:platform-team@example.com	a team — bind roles here	membership
Service account	serviceAccount:orders-api-sa@proj.iam.gserviceaccount.com	a workload inside Google Cloud	tokens from the metadata server — no password, no key
Workload identity	principal://iam.googleapis.com/.../subject/repo:acme/orders	a workload outside Google Cloud	its own provider's token, exchanged — no key

SPECIAL PRINCIPALS

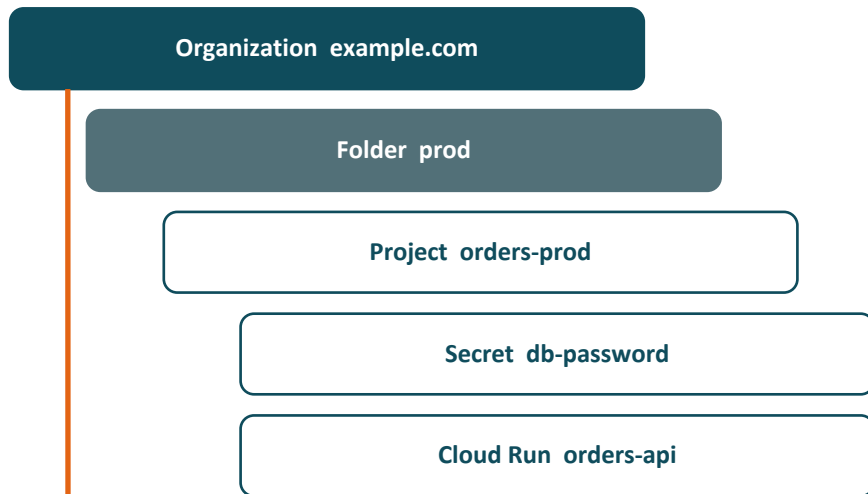
- allUsers = anyone on the internet · allAuthenticatedUsers = anyone with a Google account. Both are what 'public' means in IAM — and both are exactly what the Domain restricted sharing org policy refuses.

THE RULE

People get roles through groups; workloads get identities of their own; nothing gets a key.

ROLES, PERMISSIONS AND THE RESOURCE HIERARCHY

Permission flows down the tree — a role on the folder applies to every project in it



policy attached here is inherited by everything below

Role type	Example	Use
Basic	roles/owner, editor, viewer	sandboxes only — every permission on every service
Predefined	roles/run.invoker, roles/secretmanager.secretAccessor	the default: one job on one service
Custom	roles/orders.deployer (your bundle)	when no predefined role is narrow enough

```
effective(resource) = own policy U project U folder U org
# additive: nothing below can remove what an ancestor granted
```

THE RULE

Grant low in the tree with a predefined role — anything granted high is granted everywhere below.

READING A POLICY

A policy is a list of bindings — role, members, and sometimes a condition

WORKED EXAMPLE — A PROJECT POLICY WITH TWO OVER-GRANTS AND ONE GOOD CONDITION

```
$ gcloud projects get-iam-policy orders-prod --format=yaml
bindings:
- members: [user:ravi@example.com]
  role: roles/owner                # a person with Owner in prod
- members: [serviceAccount:orders-api-sa@orders-prod.iam.gserviceaccount.com]
  role: roles/editor              # a workload with Editor
- members: [group:platform-oncall@example.com]
  role: roles/run.admin
  condition:
    title: until-incident-closed
    expression: request.time < timestamp("2026-09-12T18:00:00Z")
etag: BwYk...
```

Reading habits

- One binding = one role + its members
- Ask per binding: narrowest role? every member still needed?
- Conditions bound a grant in time or scope — an expiring on-call grant is better than a permanent one
- Basic roles on a workload are always a finding

THE RULE

Every binding is a sentence: WHO may do WHAT, WHERE, and for how long — read it as one.

ORGANIZATION POLICY

Above IAM sit constraints — what may be granted at all, even by an Owner



WORKED EXAMPLE — THE 403 FROM LAST WEEK, AND THE OVERRIDE

```
$ gcloud run services add-iam-policy-binding hello --region asia-south1 --member=allUsers --role=roles/run.invoker
ERROR: FAILED_PRECONDITION: One or more users named in the policy do not belong to a permitted customer,
    perhaps due to an organization policy.
# the override (needs roles/orgpolicy.policyAdmin on the ORGANIZATION):
name: projects/PROJECT/policies/iam.allowedPolicyMemberDomains
spec: { rules: [ { allowAll: true } ] }
$ gcloud org-policies set-policy allow-all.yaml
```

Three constraints to know by name

- iam.allowedPolicyMemberDomains — no principals outside your domain
- iam.disableServiceAccountKeyCreation — no downloadable keys (chapter 2)
- compute.skipDefaultNetworkCreation — no auto-mode default network (landscape session)

THE RULE

IAM grants; org policy constrains what may be granted — a 403 with a correct binding means look one level up.

Start from nothing and add — a role is earned by an error message

1

Create the identity with no roles

`gcloud iam service-accounts create orders-api-sa — nothing bound`

2

Run it and read the error

the message names the missing permission exactly

3

Grant the narrowest predefined role, on the narrowest resource

then run again — repeat until it works

WORKED EXAMPLE — THE ERROR NAMES THE PERMISSION; THE GRANT NAMES THE RESOURCE

```
$ gcloud run services logs read orders-api --region asia-south1 --limit 3
PermissionDenied: 403 Permission 'secretmanager.versions.access' denied
  for resource 'projects/orders-prod/secrets/db-password/versions/latest'
$ gcloud secrets add-iam-policy-binding db-password \
  --member=serviceAccount:orders-api-sa@orders-prod.iam.gserviceaccount.com \
  --role=roles/secretmanager.secretAccessor      # on the SECRET, not the project
```

THREE RULES

- No basic role on a workload, ever
- Grant on the resource when the role allows it (secrets, services, buckets do)
- Write the reason beside the binding — the auditor will ask

THE RULE

A role is earned by a permission-denied error — if you cannot name the error it fixed, remove it.

ACTIVITY A

Ten minutes — find the three over-grants in this policy

The lab manual has a full policy dump for a fictional orders-prod project (12 bindings). In pairs: mark the three bindings that violate least privilege, name the narrower role that should replace each, and say on which resource it should be granted.

1 • Read

Each binding as a sentence: who, what, where, how long.

2 • Mark

Three bindings are wrong. One is a basic role on a workload; one is project-wide where it should be per-resource; one is a person who should be a group.

3 • Replace

Write the replacement binding — role and resource — under each.

02

Identity for workloads

The model behind what you already did: a service account per service, an identity token per call, and no keys anywhere.

01 Identity and the hierarchy

02 Identity for workloads

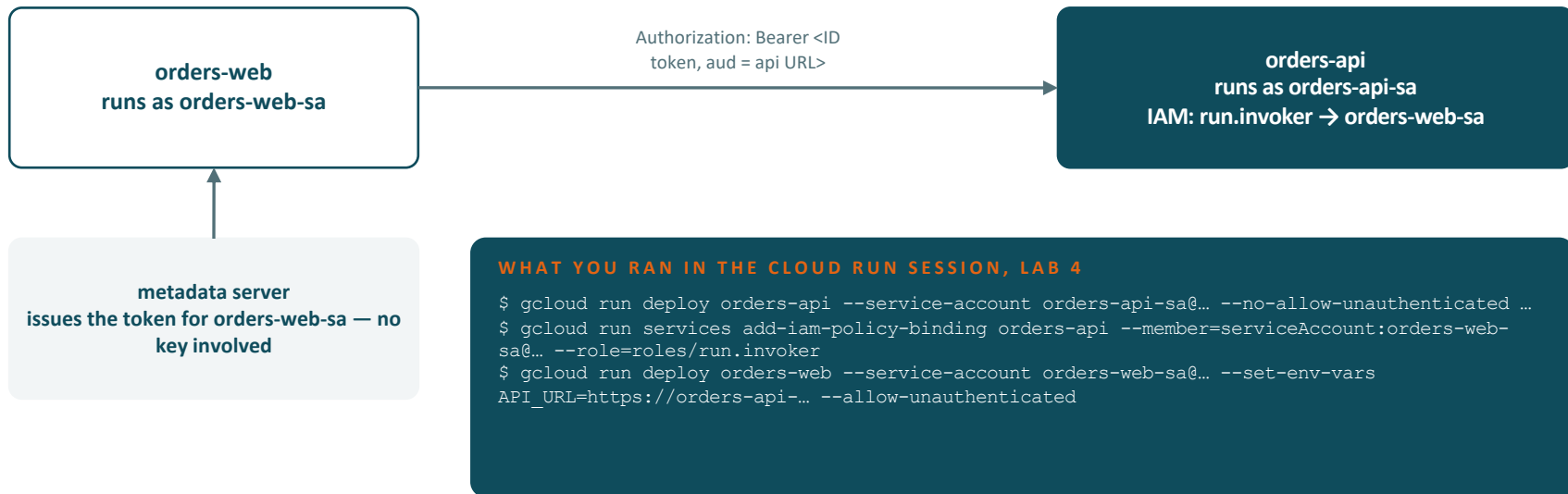
03 Secrets

04 Environment boundaries

05 Auditing access

THE CLOUD RUN SERVICE'S OWN IDENTITY

You already built this — one service account per service, one binding per caller



THE RULE

One identity per service, no roles by default, one invoker binding per caller — the pattern is already yours.

SERVICE-TO-SERVICE AUTHENTICATION AND IDENTITY TOKENS

The token is a signed statement: who I am, who I am talking to, until when

WORKED EXAMPLE — AN IDENTITY TOKEN, DECODED (SIGNATURE NOT SHOWN)

```
$ TOKEN=$(gcloud auth print-identity-token --audiences=https://orders-api-123.asia-south1.run.app)
$ echo $TOKEN | cut -d. -f2 | base64 -d 2>/dev/null | python3 -m json.tool
{
  "iss": "https://accounts.google.com",
  "aud": "https://orders-api-123.asia-south1.run.app",    # for this service only
  "email": "orders-web-sa@orders-prod.iam.gserviceaccount.com",
  "email_verified": true,
  "iat": 1757400000, "exp": 1757403600                  # valid for one hour
}
```

What Cloud Run checks, in order

- 1 Signature**
signed by Google?
- 2 Audience**
aud == my URL?
- 3 IAM**
email holds run.invoker here?

A token for one service is useless against another — the audience is the binding.

THE RULE

Identity tokens are minted, not stored: who I am and who I am for, signed by Google, gone in an hour.

ACTING AS A SERVICE ACCOUNT

Deploying 'as' an identity is itself a permission — actAs is the gate

WORKED EXAMPLE — A DEPLOYER WITHOUT ACTAS, AND THE GRANT ON THE SERVICE ACCOUNT ITSELF

```
$ gcloud run deploy orders-api --image ... --service-account orders-api-sa@orders-prod.iam.gserviceaccount.com
ERROR: (gcloud.run.deploy) PERMISSION_DENIED: Permission 'iam.serviceaccounts.actAs' denied on
service account orders-api-sa@orders-prod.iam.gserviceaccount.com (or it may not exist).
$ gcloud iam service-accounts add-iam-policy-binding orders-api-sa@orders-prod.iam.gserviceaccount.com \
--member=group:orders-deployers@example.com --role=roles/iam.serviceAccountUser
```

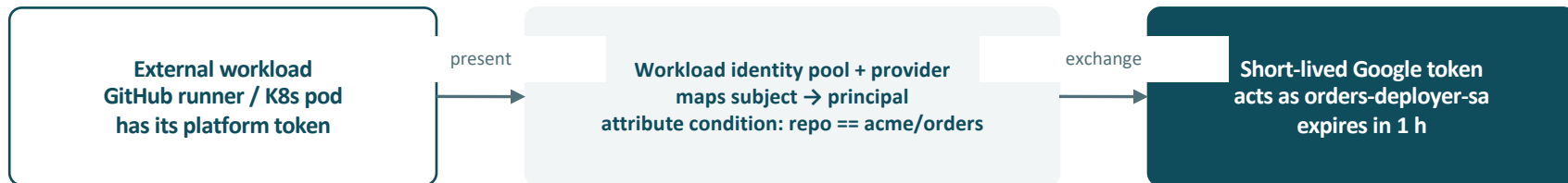
Role	Permission it carries	Lets the holder
roles/iam.serviceAccountUser	iam.serviceAccounts.actAs	deploy or run workloads as that service account
roles/iam.serviceAccountTokenCreator	iam.serviceAccounts.getAccessToken / getOpenIdToken	impersonate it: mint its tokens from the CLI
roles/iam.serviceAccountKeyAdmin	iam.serviceAccountKeys.create	download a key — block it with the org policy

THE RULE

Running as an identity is a permission on that identity — grant Service Account User per service account, never per project.

WORKLOAD IDENTITY

Outside Google Cloud, the same idea — exchange a token you have, never download a key



THE OLD WAY

- A service-account JSON key: a credential that never expires, stored in the CI system, copied into laptops, committed by accident
- Block it: org policy `iam.disableServiceAccountKeyCreation`

THE FEDERATED WAY

- Nothing to store: the workload's own token is exchanged for a one-hour Google token
- Attribute conditions bound which external identities may map — repo, branch, cluster namespace

THE RULE

A downloadable key is a password that never expires — federate identity instead, and forbid keys by policy.

LAB 1

Run bookshelf-api on an identity with nothing — then earn each permission

LAB 1 · LAB MANUAL

Open the lab manual, Lab 1. Every command, console path and expected output is there — nothing is typed from this slide.

- Create a fresh service account with no roles; redeploy bookshelf-api as it — it still serves
- Decode the identity token bookshelf-web presents and find the audience
- Try to deploy as an account you lack actAs on; read the error, grant Service Account User, retry
- List every role the two service accounts hold — the answer should be one binding

About 25 minutes
Kit: gcp-iam-labs.zip
Cloud Shell only — no code

THE RULE

Nothing by default, one binding per need, and a token you can read — identity you can explain line by line.

03

Secrets

Why a key in an environment variable is a defect, and how Secret Manager, versions and Cloud Run references fix it without code.

01 Identity and the hierarchy

02 Identity for workloads

03 Secrets

04 Environment boundaries

05 Auditing access

WHY AN API KEY IN AN ENVIRONMENT VARIABLE IS A DEFECT

It is visible to everyone who can describe the service, forever, in every revision

WORKED EXAMPLE — A PASSWORD IN AN ENV VAR, READ BY A VIEWER

```
$ gcloud run services describe orders-api --region asia-south1 --  
format=yaml (spec.template.spec.containers[0].env) '  
env:  
- name: DB_HOST  
  value: 10.10.4.5  
- name: DB_PASSWORD  
  value: Tr0ub4dor&3          # readable by every roles/viewer, in every  
revision, forever  
$ gcloud run revisions list --service orders-api --region asia-south1 | wc -l  
14                          # fourteen copies of that password in history
```

Where it leaks

- describe / console — plain text to any viewer
- every revision keeps it — rotation removes nothing
- crash dumps and framework logs echo the environment
- same value copied to dev, staging, prod

	Env var	Secret Manager
Who can read it	any project viewer	only principals bound on the secret
Rotation	new revision, old value in history	new version; old version disabled
Access log	none	every access, with the caller

THE RULE

A secret needs its own access control, its own history and its own log — an environment variable has none of the three.

SECRET MANAGER — VERSIONS, ROTATION AND ACCESS CONTROL

A secret is a name with access control; values live in versions you enable, disable, destroy

WORKED EXAMPLE — CREATE, TWO VERSIONS, ROTATE, DISABLE, GRANT ON THE SECRET

```
$ gcloud secrets create db-password --replication-policy=automatic
$ printf 'Tr0ub4dor&3' | gcloud secrets versions add db-password --data-file=- #
version 1
$ printf 'correct-horse-battery' | gcloud secrets versions add db-password --data-file=-
# version 2
$ gcloud secrets versions access latest --secret=db-password
correct-horse-battery
$ gcloud secrets versions disable 1 --secret=db-password
$ gcloud secrets versions list db-password
NAME  STATE  CREATED
2     enabled 2026-09-09T09:12:41
1     disabled 2026-09-09T09:11:05
$ gcloud secrets add-iam-policy-binding db-password --member=serviceAccount:orders-api-sa@...
--role=roles/secretmanager.secretAccessor
```

Version states

- enabled — readable
- disabled — refused, reversible: your rollback for a bad rotation
- destroyed — gone for good
- latest — alias for the newest enabled version

IAM attaches to the secret. `secretAccessor` reads payloads; `secretVersionManager` rotates; admin does both — nobody gets admin on the project.

THE RULE

Rotate by adding a version and disabling the old one; grant access on the secret, never on the project.

MOUNTING A SECRET VERSUS READING IT AT RUNTIME

Three ways in — and only one of them sees a rotation without a redeploy

Method	How the container sees it	Resolved when	A new 'latest' version is seen
Environment variable	\$DB_PASSWORD	before the instance starts, once	only by a new instance — pin a version
Mounted file	/secrets/db-password	on every read of the file	on the next read — no redeploy
Runtime API call	client library in the app	whenever the code asks	whenever the code asks — code now depends on Secret Manager

WHAT THE CLOUD RUN DOCUMENTATION SAYS ABOUT RESOLUTION TIMING (VERIFIED 2026-09-09)

```
# env var: value fetched before the instance starts — one fetch per instance
Cloud Run retrieves the value of the secret prior to starting the instance.
# file mount: fetched on read — 'latest' means latest
When reading a volume, Cloud Run always fetches the secret value from Secret Manager.
# guidance: env var + latest = stale-but-looks-current → pin a version, or mount
```

THE RULE

Mount a file when the value may rotate; inject an env var only with a pinned version; keep Secret Manager out of the code.

REFERENCING SECRETS FROM CLOUD RUN WITHOUT CODE CHANGES

One flag, two spellings — the container sees a variable or a file, never the API

WORKED EXAMPLE — THE SAME SECRET AS A VARIABLE AND AS A FILE; THE REVISION STORES A REFERENCE, NOT THE VALUE

```
# as an environment variable, pinned to version 2
$ gcloud run services update orders-api --region asia-south1 --update-secrets=DB_PASSWORD=db-password:2
# as a file, tracking latest
$ gcloud run services update orders-api --region asia-south1 --update-secrets=/secrets/db-password=db-password:latest
$ gcloud run services describe orders-api --region asia-south1 --
format='yaml (spec.template.spec.volumes,spec.template.spec.containers[0].env) '
env:
- name: DB_PASSWORD
  valueFrom: { secretKeyRef: { key: '2', name: db-password } }      # a reference — no value in the revision
volumes:
- name: db-password-... secret: { secretName: db-password, items: [{ key: latest, path: db-password }] }
```

WHEN THE BINDING IS MISSING

- Revision fails to start: 'Permission denied on secret: projects/.../secrets/db-password/versions/2 for Revision service account orders-api-sa@...' — grant secretAccessor on the secret and redeploy

THE RULE

The revision stores a reference, the container sees a variable or a file, the code never learns Secret Manager exists.

Three buckets — env var, Secret Manager, and 'not stored at all'

Configuration → env var

- differs per environment, not sensitive
- API_URL, DEPOT, LOG_LEVEL, feature flags
- visible in describe — that is fine
- versioned with the revision

Secret → Secret Manager

- disclosure causes harm
- DB password, third-party API key, signing key
- referenced with --set-secrets
- rotated by version, logged on access

Not stored at all

- the platform already provides it
- own identity, project id, Google API tokens
- metadata server, not a key file
- storing it is the bug

The test: if this value appeared in a screenshot on a slide, would someone have to act? Yes → secret. No → config. Is it Google's own credential? → do not store it.

THE RULE

Config is visible and per-environment; a secret is referenced and logged; a platform credential is never stored.

LAB 2

Give bookshelf-api a managed secret — as a file and as a variable — then rotate it

LAB 2 · LAB MANUAL

Open the lab manual, Lab 2. Every command, console path and expected output is there — nothing is typed from this slide.

- Create a secret with version 1; grant secretAccessor to bookshelf-api-sa only — on the secret
- Deploy two revisions: one with the secret mounted at /secrets, one with it as an env var pinned to 1
- Add version 2; read both — one changes, one does not; disable version 1 and read again
- Try to read the secret as the other service account and read the denial

About 30 minutes
Kit: gcp-iam-labs.zip
Cloud Shell only — no code

THE RULE

Two revisions, one rotation, one denial — you saw which method follows a rotation and who may read what.

04

Environment boundaries

Development, staging and production as trust zones — projects, folders and ingress as the walls between them.

01 Identity and the hierarchy

02 Identity for workloads

03 Secrets

04 Environment boundaries

05 Auditing access

ENVIRONMENT BOUNDARIES

Development, staging and production are trust zones — not names, not labels

	Development	Staging	Production
Who deploys	any engineer	the pipeline	the pipeline only
Who reads logs / config	any engineer	engineers	on-call; break-glass for the rest
Data	synthetic	realistic, anonymised	real — customer data
Secrets	dev values, low value	staging values	production values, rotated, access-logged
Audit expectation	none	review on incident	every access reviewable
Mistakes cost	a rebuild	a delayed release	customers, money, the auditor's attention

THE SINGLE-PROJECT ANTI-PATTERN

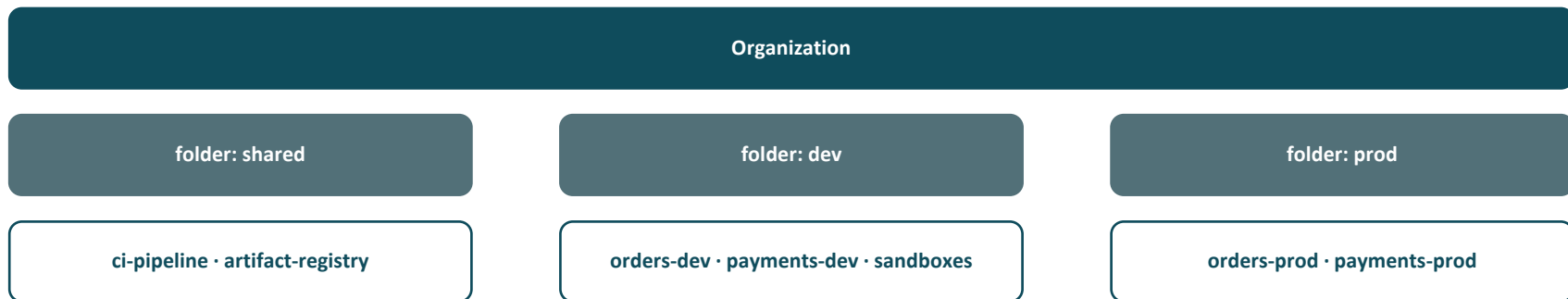
- orders-api-dev and orders-api-prod in one project share one IAM policy, one secret store, one audit trail and one blast radius — a dev mistake is a prod

THE RULE

An environment is a set of people, data and rules — three environments means three places with three policies.

PROJECT AND FOLDER STRUCTURE AS A BLAST-RADIUS CONTROL

One project per environment per workload; folders carry the policy you set once



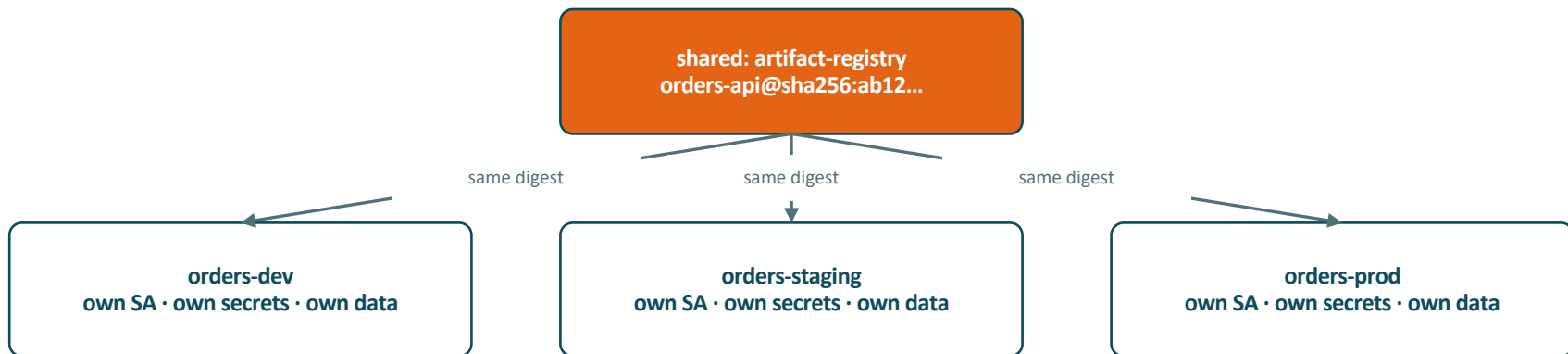
Level	Owns
Organization	org policies, the admin groups, billing accounts
Folder	environment-wide IAM and policy overrides, audit-log config — set once, inherited
Project	APIs, quota, IAM for one workload in one environment, its own audit trail

THE RULE

A project bounds the damage; a folder carries the rules — one project per environment per workload is the default, not the exception.

WHAT CROSSES A BOUNDARY AND WHAT DOES NOT

Images cross; identities, secrets and data stay home



Crosses the boundary	Stays inside it
the image (one digest, promoted)	service accounts — one per environment
the pipeline identity, with a narrow deploy role per target	secrets — same name, different value, different project
	data — never down from prod without anonymising

THE RULE

The image is the only thing that is identical across environments — if something else is, it is a leak waiting to happen.

INGRESS SETTINGS AND INTERNAL-ONLY SERVICES

Ingress is a network boundary; IAM is an identity boundary — production uses both

	ingress = all	ingress = internal	internal + load balancing
no invoker binding (authenticated only)	reachable, every caller refused	reachable only from the VPC; refused	reachable from VPC and your LB; refused
invoker → one service account	internet can knock; one identity enters	only VPC callers can knock; one identity enters — the private API	as internal, plus a front door you control
invoker → allUsers / no IAM check	public — the web page	internal-public: any VPC workload	public through your LB with Cloud Armor

```
$ gcloud run services update orders-api --region asia-south1 --ingress internal
$ gcloud run services describe orders-api --region asia-south1 --
format='value(metadata.annotations."run.googleapis.com/ingress") '
internal
```

THE RULE

Ingress says where a request may come from, IAM says who may send it — a private service sets both.

PROMOTING THE SAME IMAGE ACROSS ENVIRONMENTS

Same digest, different identity, secrets and ingress — that is a promotion

WORKED EXAMPLE — ONE DIGEST, THREE ENVIRONMENTS; ONLY THE PROJECT, IDENTITY, INGRESS AND CONFIG CHANGE

```
IMG=asia-south1-docker.pkg.dev/shared-registry/orders/orders-api@sha256:ab12... # identical below
$ gcloud run deploy orders-api --project orders-dev      --image $IMG --service-account orders-api-sa@orders-dev.iam.gserviceaccount.com \
  --set-secrets=/secrets/db-password=db-password:latest --ingress all      --set-env-vars LOG_LEVEL=debug
$ gcloud run deploy orders-api --project orders-staging --image $IMG --service-account orders-api-sa@orders-staging.iam.gserviceaccount.com \
  --set-secrets=/secrets/db-password=db-password:latest --ingress internal --set-env-vars LOG_LEVEL=info
$ gcloud run deploy orders-api --project orders-prod    --image $IMG --service-account orders-api-sa@orders-prod.iam.gserviceaccount.com \
  --set-secrets=/secrets/db-password=db-password:latest --ingress internal --set-env-vars LOG_LEVEL=info
```

WHAT A PROMOTION CHANGES

- project · service account · secret values (same names) · ingress · config env vars

WHAT IT MUST NEVER CHANGE

- the image digest — a rebuild per environment means staging proved nothing

THE RULE

Promote the digest, swap the surroundings — Session D turns these three commands into a pipeline.

ACTIVITY B

Fifteen minutes — draw the boundaries for Bookshelf across three environments

On the template in the lab manual: place bookshelf-web and bookshelf-api in dev, staging and prod; mark every service account, every secret, the ingress of each service, and the one shared resource. Then answer: what can a leaked dev credential reach?

1 • Projects and folders

Draw the folder/project tree. Where does the shared registry live? Where does the org-policy override live?

2 • Identities and secrets

Name every service account (project-qualified) and every secret, per environment. Six of each.

3 • Blast radius

A dev service-account token leaks. List exactly what it can reach — it should be a short list.

05

Auditing access

Two questions an auditor asks — who can read the secret, and who did — and the two tools that answer them.

01 Identity and the hierarchy

02 Identity for workloads

03 Secrets

04 Environment boundaries

05 Auditing access

AUDITING ACCESS

Admin Activity is always on; Data Access — the one that shows reads — is off by default

Log	Records	Default	Example entry
Admin Activity	changes to config and policy	on, cannot be turned off	SetIamPolicy on secret db-password by ravi@
System Event	Google-initiated changes	on	Cloud Run scaled a revision
Data Access	reads and writes of data	OFF — enable per service	AccessSecretVersion db-password/2 by orders-api-sa@

WORKED EXAMPLE — ENABLING DATA ACCESS LOGS FOR SECRET MANAGER (DO THIS ON THE PROD FOLDER IN REAL LIFE)

```
$ gcloud projects get-iam-policy orders-prod --format=yaml > policy.yaml # then add under the top level:
auditConfigs:
- service: secretmanager.googleapis.com
  auditLogConfigs:
    - logType: DATA_READ
    - logType: DATA_WRITE
$ gcloud projects set-iam-policy orders-prod policy.yaml
# console: IAM & Admin → Audit Logs → Secret Manager API → Data Read, Data Write
```

THE RULE

Admin Activity tells you who changed the lock; only Data Access tells you who opened it — and it is off until you turn it on.

WHO CAN READ THE SECRET, AND WHO DID

Two questions, two tools — a policy analysis for 'can', an audit-log query for 'did'

WHO CAN — POLICY ANALYZER WALKS INHERITANCE AND GROUPS

```
$ gcloud asset analyze-iam-policy --project=orders-prod \
  --full-resource-name=//secretmanager.googleapis.com/projects/orders-prod/secrets/db-password \
  --permissions=secretmanager.versions.access
identity: serviceAccount:orders-api-sa@orders-prod.iam.gserviceaccount.com   via roles/secretmanager.secretAccessor on the secret
identity: user:ravi@example.com                                             via roles/owner on projects/orders-prod   # inherited
```

WHO DID — THE DATA ACCESS AUDIT LOG, ONE LINE PER READ

```
$ gcloud logging read 'protoPayload.serviceName="secretmanager.googleapis.com" AND
  protoPayload.methodName="google.cloud.secretmanager.v1.SecretManagerService.AccessSecretVersion"' --limit 5 \
  --format='table(timestamp, protoPayload.authenticationInfo.principalEmail, protoPayload.resourceName)'
2026-09-09T09:41:07Z   orders-api-sa@orders-prod.iam.gserviceaccount.com   projects/.../secrets/db-password/versions/2
2026-09-09T09:12:55Z   ravi@example.com               projects/.../secrets/db-password/versions/2
```

THE RULE

'Can' is a policy analysis across the whole tree; 'did' is the Data Access log — and both must be answerable before anyone asks.

WHAT AN AUDITOR ASKS

Four questions, and the artefact that answers each — none of them is a screenshot

The question	The evidence	Produced by
Who can read the secret?	Policy Analyzer result: every identity, the binding it comes through	gcloud asset analyze-iam-policy
Who did read it, and when?	Data Access log entries for AccessSecretVersion, exported with retention	Data Access logging on + a log sink (Session A)
How is it rotated?	the version list with creation dates and states	gcloud secrets versions list
Who approved the current access?	the change record referencing each binding	your change process (Session B)

BEFORE ANY REVIEW

- Data Access logs enabled on the prod folder · a log sink with retention · Cloud Asset API enabled · every binding traceable to a change record

THE RULE

Evidence is an export with a timestamp — if the log was off, the evidence never existed.

LAB 3

Bring it together — least-privilege identities, a managed secret, and the two audit answers

LAB 3 · LAB MANUAL

Open the lab manual, Lab 3. Every command, console path and expected output is there — nothing is typed from this slide.

- Both services on their own accounts with exactly the bindings they need — list and justify each
- The secret mounted on bookshelf-api at /secrets, version 2 live, version 1 disabled; api on internal ingress
- Data Access logging on for Secret Manager; read the secret once yourself, then find both callers in the log
- Run Policy Analyzer for the secret: the inherited Owner you never bound is the finding

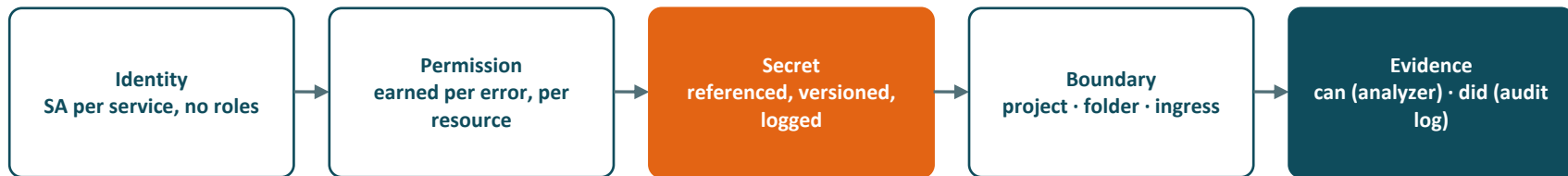
About 25 minutes
Kit: gcp-iam-labs.zip
Cloud Shell only — no code
Leaves the state Sessions D and B start from

THE RULE

Who can, who did, and why each binding exists — the state your service should be in before anyone audits it.

THE WHOLE PATH IN ONE PICTURE

Identity → permission → secret → boundary → evidence



You leave with	
a policy you can read as sentences	and a habit: every binding has an error it fixed
a secret the code never learned about	mounted, rotated by version, disabled before destroyed
a three-environment layout on paper	and the blast-radius answer for a leaked dev token
two commands for the auditor	Policy Analyzer for can; Data Access log for did

The walls are up. The next sessions move things through them.

CI/CD promotes the same digest through the environments you drew, under a pipeline identity that holds only actAs on the deployers. Observability turns the logs you enabled today into dashboards and alerts. Incident response starts from the state Lab 3 left behind.

Leave the services running. Kit: [gcp-iam-labs.zip](#) · Lab manual: [gcp-iam-lab-manual.pdf](#)